



Sorting (Pengurutan)

Bubble Sort • Selection Sort • Analisis Kompleksitas

Algoritma Pemrograman • IF25-11001 • 2 SKS Teori

Institut Teknologi Sumatera — Teknik Informatika

Review Pertemuan 11

Searching — pondasi untuk sorting

Linear Search = $O(n)$

Cek satu-per-satu, tidak perlu terurut. Sederhana tapi lambat untuk data besar.



Binary Search = $O(\log n)$

Bagi dua setiap langkah. HARUS terurut dulu. 1 miliar data = 30 langkah!

Hari ini: MENGURUTKAN data!

Bubble Sort & Selection Sort — dua algoritma sorting fundamental $O(n^2)$.

Capaian Pembelajaran Pertemuan 12

Sub-CPMK 12.1

C2 – Menjelaskan

Menjelaskan konsep mengurutkan dan membandingkan Bubble Sort vs Selection Sort

CPMK0609

Sub-CPMK 12.2

C3 – Menerapkan

Melakukan trace Bubble Sort step-by-step dan menghitung jumlah swap

CPMK0609

Sub-CPMK 12.3

C3 – Menerapkan

Melakukan trace Selection Sort dan menganalisis kompleksitas $O(n^2)$

CPMK0609

Outline Pertemuan Hari Ini

	00-10	Review dan Hook	Recap searching + motivasi sorting
	10-30	Bubble Sort	Konsep, pseudocode, trace, analisis
	30-50	Selection Sort	Konsep, pseudocode, trace, analisis
	50-60	Perbandingan	Bubble vs Selection, stabilitas
	60-75	Workshop Trace	Trace kedua algoritma di kertas
	75-85	Algorithm Detective	Bug: wrong swap & boundary error
	85-95	PBL#2 Final Check	Submit Data Analyzer lengkap
	95-100	Exit Ticket dan Tugas	Assessment + homework



Pertanyaan Pemantik

*"Guru membagikan 40 lembar ujian.
Bagaimana cara mengurutkan berdasarkan nama A–Z?"*

*"Spotify mengurutkan playlist berdasarkan popularitas.
Bagaimana caranya dari miliaran lagu?"*

*"Apakah semua cara mengurutkan sama cepatnya?
Atau ada cara yang lebih efisien?"*

Bubble Sort

Bandingkan dan swap elemen BERTETANGGA — elemen besar "menggelembung" ke akhir

Pass 1: arr = {5, 3, 8, 1, 4} — bandingkan setiap pasangan bertetangga

Step 1: 5 > 3?	YA → swap	→ {3,5,8,1,4}
Step 2: 5 > 8?	TIDAK	→ {3,5,8,1,4}
Step 3: 8 > 1?	YA → swap	→ {3,5,1,8,4}
Step 4: 8 > 4?	YA → swap	→ {3,5,1,4,8}

Setelah Pass 1: elemen TERBESAR (8) sudah di posisi terakhir!

Pseudocode Bubble Sort

```
UNTUK i ← 0 SAMPAI N-2
  UNTUK j ← 0 SAMPAI N-2-i
    JIKA arr[j] > arr[j+1] MAKA
      temp ← arr[j]
      arr[j] ← arr[j+1]
      arr[j+1] ← temp
    AKHIR JIKA
  AKHIR UNTUK
AKHIR UNTUK
```

Karakteristik

- Bandingkan elemen BERTETANGGA
- Swap jika urutan salah
- Setiap pass: 1 elemen terbesar "fix"
- N-1 pass total untuk N elemen
- Kompleksitas: $O(n^2)$ semua kasus
- Stabil: elemen sama tetapurut

Trace: Bubble Sort

arr = {5, 3, 8, 1, 4} — urutkan ascending

Pass	Perbandingan & Swap	Array Setelah Pass	Swap	Elemen Fix
Pass 1 (i=0)	5>3→swap, 5<8, 8>1→swap, 8>4→swap	{3, 5, 1, 4, 8}	3	8
Pass 2 (i=1)	3<5, 5>1→swap, 5>4→swap	{3, 1, 4, 5, 8}	2	5,8
Pass 3 (i=2)	3>1→swap, 3<4	{1, 3, 4, 5, 8}	1	4,5,8
Pass 4 (i=3)	1<3	{1, 3, 4, 5, 8}	0	semua

Total: 4 pass, 10 perbandingan, 6 swap. Kompleksitas: $O(n^2) = O(5^2) = O(25)$

Mengapa $O(n^2)$?

Outer loop: N-1 pass = 4 kali

Inner loop setiap pass: N-1-i perbandingan

Pass 1: 4 perbandingan, Pass 2: 3, Pass 3: 2, Pass 4: 1

Total = 4+3+2+1 = $N(N-1)/2 = 5 \times 4 / 2 = 10$ perbandingan

Untuk N elemen: $N(N-1)/2 \approx n^2/2 = O(n^2)$

Selection Sort

Cari elemen MINIMUM, lalu TUKAR ke posisi yang benar

Pass 1: arr = {5, 3, 8, 1, 4} – cari MINIMUM dari seluruh array

Scan i=0..4

Cari minimum: min=1 di indeks 3

Swap

Tukar arr[0]=5 dengan arr[3]=1

Hasil

{1, 3, 8, 5, 4} — elemen terkecil sudah di posisi 0!

Pseudocode Selection Sort

```
UNTUK i ← 0 SAMPAI N-2
  minIdx ← i
  UNTUK j ← i+1 SAMPAI N-1
    JIKA arr[j] < arr[minIdx] MAKA
      minIdx ← j
  AKHIR JIKA
  AKHIR UNTUK
  SWAP arr[i] ↔ arr[minIdx]
AKHIR UNTUK
```

Karakteristik

- Cari elemen MINIMUM di sisa array
- Swap minimum ke posisi yang benar
- Setiap pass: 1 elemen terkecil "fix"
- Jumlah swap: tepat N-1 (efisien!)
- Kompleksitas: $O(n^2)$ semua kasus
- Tidak stabil (bisa ubah urutan sama)

Trace: Selection Sort

arr = {5, 3, 8, 1, 4} — urutkan ascending

Pass	Unsorted Range	minIdx	min Value	Swap	Array Setelah
i=0	[0..4]	3	1	arr[0]↔arr[3]	{1, 3, 8, 5, 4}
i=1	[1..4]	1	3	tidak swap	{1, 3, 8, 5, 4}
i=2	[2..4]	4	4	arr[2]↔arr[4]	{1, 3, 4, 5, 8}
i=3	[3..4]	3	5	tidak swap	{1, 3, 4, 5, 8}

Total: 4 pass, 10 perbandingan, hanya 2 swap! (Bubble Sort: 6 swap)

Selection vs Bubble: Jumlah Swap

Selection Sort: selalu $N-1$ swap (maksimum). Cari min dulu, baru swap SEKALI.

Bubble Sort: bisa 0 sampai $N(N-1)/2$ swap. Swap SETIAP kali ada pasangan salah.

Jika swap mahal (data besar), Selection Sort lebih efisien!

Jika data hampir terurut, Bubble Sort bisa sangat cepat (early termination).

Perbandingan: Bubble Sort vs Selection Sort

Keduanya $O(n^2)$ — tapi punya perbedaan penting!

Aspek	Bubble Sort	Selection Sort
Cara Kerja	Swap elemen BERTETANGGA	Cari MIN, swap ke posisi benar
Perbandingan	$N(N-1)/2$	$N(N-1)/2$
Swap (worst)	$N(N-1)/2$ (banyak!)	$N-1$ (sedikit!)
Stabil?	YA (elemen sama tetap urut)	TIDAK (bisa ubah urutan sama)
Best Case	$O(n)$ jika sudah terurut*	$O(n^2)$ selalu
Memori	In-place $O(1)$	In-place $O(1)$
Cocok untuk	Data kecil, hampir terurut	Data kecil, swap mahal

* Bubble Sort $O(n)$ best case memerlukan optimasi early termination (flag swapped)

Optimasi: tambahkan flag swapped $\leftarrow$ false. Jika setelah 1 pass tidak ada swap, array sudah terurut $\rightarrow$ BERHENTI!

Workshop: Trace Sorting

Kerjakan di kertas — 10 menit

Soal 1: Bubble Sort

Diberikan:

$\text{arr} = \{7, 2, 9, 4, 1\}$

$N = 5$

Pertanyaan:

a) Trace Bubble Sort selengkapnya.

Untuk SETIAP pass, tulis:

- Perbandingan yang dilakukan
- Swap yang terjadi
- Array setelah pass selesai

b) Berapa TOTAL perbandingan?

c) Berapa TOTAL swap?

Soal 2: Selection Sort

Diberikan:

$\text{arr} = \{7, 2, 9, 4, 1\}$

$N = 5$

Pertanyaan:

a) Trace Selection Sort selengkapnya.

Untuk SETIAP pass, tulis:

- Range pencarian minimum
- minIdx dan min value
- Swap yang terjadi
- Array setelah pass selesai

b) Berapa TOTAL swap?

c) Bandingkan swap Bubble vs Selection!

Jawaban Workshop

arr = {7, 2, 9, 4, 1}

Bubble Sort

P1: 7>2→swap, 7<9, 9>4→swap, 9>1→swap → {2, 7, 4, 1, 9} 3sw

P2: 2<7, 7>4→swap, 7>1→swap → {2, 4, 1, 7, 9} 2sw

P3: 2<4, 4>1→swap → {2, 1, 4, 7, 9} 1sw

P4: 2>1→swap → {1, 2, 4, 7, 9} 1sw

Total: 10 perbandingan, 7 swap

Selection Sort

P1: scan[0..4] min=1@idx4, swap arr[0]↔arr[4] → {1, 2, 9, 4, 7}

P2: scan[1..4] min=2@idx1, no swap → {1, 2, 9, 4, 7}

P3: scan[2..4] min=4@idx3, swap arr[2]↔arr[3] → {1, 2, 4, 9, 7}

P4: scan[3..4] min=7@idx4, swap arr[3]↔arr[4] → {1, 2, 4, 7, 9}

Total: 10 perbandingan, 3 swap

Perbandingan Hasil

	Perbandingan	Swap	Pemenang
Bubble Sort	10	7	
Selection Sort	10	3	✓ Lebih sedikit swap!

Kesimpulan: Perbandingan sama (keduanya $O(n^2)$), tapi Selection Sort lebih hemat swap.

Jika data besar dan swap mahal, Selection Sort lebih baik. Jika data hampir terurut, Bubble Sort + early termination.



Algorithm Detective

Temukan bug pada sorting berikut!

Kasus 1: Wrong Swap (Bubble)

```
JIKA arr[j] > arr[j+1] MAKA  
  arr[j] ← arr[j+1]  
  arr[j+1] ← arr[j]
```

Bug: swap TANPA temp!
 $arr[j] = arr[j+1] \rightarrow$ nilai $arr[j]$ hilang!
Kedua elemen jadi SAMA.

Contoh: $arr[j]=5, arr[j+1]=3$
 $arr[j] \leftarrow 3 \rightarrow arr[j+1] \leftarrow 3 \rightarrow \{3,3\}!$

Fix: gunakan variabel temp!
 $temp \leftarrow arr[j]$
 $arr[j] \leftarrow arr[j+1]$
 $arr[j+1] \leftarrow temp$

Kasus 2: Boundary Error (Selection)

```
UNTUK i ← 0 SAMPAI N-1  
  minIdx ← i  
  UNTUK j ← i SAMPAI N-1  
    ...
```

Bug 1: Outer loop sampai N-1
Seharusnya N-2! (elemen terakhir
otomatis terurut)

Bug 2: Inner loop mulai dari i
Seharusnya i+1! (tidak perlu
bandingkan elemen dengan dirinya)

Fix:
UNTUK i ← 0 SAMPAI N-2
 UNTUK j ← i+1 SAMPAI N-1



Aturan Emas Sorting

Hindari bug yang paling umum

1

Swap HARUS pakai temp

Tanpa variabel temp, salah satu nilai hilang. Selalu: $\text{temp} = a$, $a = b$, $b = \text{temp}$.

```
temp ← a; a ← b; b ← temp
```

2

Outer loop: 0 sampai N-2

N elemen butuh N-1 pass. Elemen terakhir otomatis terurut setelah N-1 pass.

```
UNTUK i ← 0 SAMPAI N-2
```

3

Bubble inner: j sampai N-2-i

Setiap pass, elemen terakhir sudah fix. Kurangi range inner loop setiap pass.

```
j ← 0 SAMPAI N-2-i
```

4

Selection inner: j mulai i+1

Tidak perlu bandingkan elemen dengan dirinya sendiri. Mulai dari i+1.

```
j ← i+1 SAMPAI N-1
```

PBL#2: Data Analyzer — DEADLINE!

Submit Data Analyzer lengkap hari ini

Checklist Final PBL#2 Data Analyzer

- ☐ Input array N elemen dari user M1
- ☐ Statistik: total, rata-rata, maks, min, counting M1
- ☐ Linear Search: cari target, return indeks/-1 M2
- ☐ Binary Search: sort dulu, cari target M2
- ☐ Bubble Sort: urutkan array ascending M3
- ☐ Selection Sort: urutkan array ascending M3
- ☐ Tampilkan array sebelum & sesudah sort M3

Timeline PBL#2 — SELESAI!

P9-P10

Statistik array (total, rata, maks, min)

Milestone 1 

P11

Searching: Linear & Binary Search

Milestone 2 

P12

Sorting: Bubble & Selection Sort

Milestone 3 

HARI INI

SUBMIT Data Analyzer lengkap!

 **DEADLINE!**

Ringkasan Pertemuan 12

Bubble Sort: swap bertetangga

Bandingkan pasangan adj, swap jika salah. Stabil, $O(n^2)$, banyak swap.

Selection Sort: cari minimum

Cari min di sisa array, swap ke posisi benar. Tidak stabil, $O(n^2)$, sedikit swap.

Keduanya $O(n^2)$

Perbandingan sama: $N(N-1)/2$. Perbedaan utama: jumlah swap dan stabilitas.



Swap HARUS pakai temp!

Tanpa temp, nilai hilang. Pola: temp=a, a=b, b=temp. Selalu 3 baris.

PBL#2 Complete!

Data Analyzer: statistik + searching + sorting. Submit hari ini!

Exit Ticket — Pertemuan 12

10 soal • 15 menit • kerjakan di kertas

1

Apa perbedaan utama Bubble Sort dan Selection Sort? (cara kerja)

2

arr={6,1,4,3}. Trace Bubble Sort Pass 1. Array setelahnya?

3

Kenapa swap HARUS pakai variabel temp? Beri contoh tanpa temp.

4

arr={6,1,4,3}. Trace Selection Sort Pass 1. Swap apa?

5

Array 10 elemen. Berapa TOTAL perbandingan Bubble Sort?

6

Apa artinya sorting "stabil"? Mana yang stabil: Bubble/Selection?

7

arr={1,2,3,4,5} sudah terurut. Berapa swap Bubble Sort? Selection?

8

Bug: inner loop $j \leftarrow 0$ TO $N-1$ (tanpa $-i$). Apa dampaknya?

9

Kapan Selection Sort lebih baik dari Bubble Sort? Jelaskan!

10

arr={3,1,4}. Trace lengkap Bubble Sort sampai terurut.

Tugas Mingguan — Sorting

Deadline: sebelum Pertemuan 13 • Kerjakan di kertas

Tugas A: Individual — Trace dan Analisis Sorting

Soal 1: arr = {12, 5, 18, 3, 9, 15, 1}. N=7.

- (a) Trace Bubble Sort LENGKAP (semua pass). Hitung total perbandingan dan swap.
- (b) Trace Selection Sort LENGKAP. Hitung total perbandingan dan swap.
- (c) Bandingkan hasilnya. Mana yang lebih efisien untuk array ini?

Soal 2: arr = {1, 2, 3, 4, 5} (sudah terurut).

- (a) Trace Bubble Sort. Berapa swap? Berapa pass sebelum berhenti (dengan early termination)? Buat pseudocode lengkapnya
- (b) Trace Selection Sort. Berapa swap?

Soal 3: Berapa TOTAL perbandingan untuk Bubble Sort dan Selection Sort jika $N=100$? Gunakan rumus $N(N-1)/2$.

Tugas B: Kelompok — PBL#2 Final Submit!

Submit Data Analyzer LENGKAP: input array, statistik, searching (linear + binary), sorting (bubble + selection). Sertakan pseudocode lengkap dan trace table untuk setiap fitur.

Referensi Pertemuan 12

Buku

1. Deitel — C++ How to Program, Bab 7.8: Sorting Arrays
2. Gaddis — Starting Out with C++, Bab 8: Sorting
3. Cormen (CLRS) — Introduction to Algorithms, Bab 2: Insertion Sort

Online

1. visualgo.net/sorting — Visualisasi sorting interaktif
2. cs50.harvard.edu — Lecture 3: Sorting Algorithms
3. [geeksforgeeks.org/sorting-algorithms](https://www.geeksforgeeks.org/sorting-algorithms) — Sorting Tutorial



Terima Kasih

IF25-11001 Algoritma Pemrograman

Pertemuan 12: Sorting (Pengurutan)

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026